SpinDoctor 3.1.2 — Hand Guide

This guide explains how to use SpinDoctor, written for people using it rather than developing it. Every example command in this guide was run on the final 3.1.2 build before release. The results are in `validation/3.1.2/handguide-examples.json` in the release folder.

---

1. Quick start

Launch

1. Open the `SpinDoctor` folder inside the release folder.
2. Double-click **`BOOT.cmd`**. It checks the carrier image (`cartridge\SpinDoctor-3.1.2.tiff`), unpacks the program into `SpinDoctor\workspace`, and opens the terminal in your web browser. Use `BOOT.cmd gif` if you want to load from the GIF carrier instead.
3. Leave the black console window open while you work. Closing it stops the terminal.

Your first circuit. Type these lines into the terminal one at a time:

```
q.circuit new bell 2
q.circuit add h 0
q.circuit add cx 0 1
q.circuit run shots 1000 basis z
```

The run produces a histogram with roughly half `00` and half `11`. That is a Bell pair.

Your first report. Every command saves a report automatically. Use the report buttons in the interface to download **TXT**, **PDF** or **CSV**. `q.history list` shows the recent reports.

---

2. Core concepts

Concept	What it means
Workspace	The folder where your session lives (`SpinDoctor\workspace`). It holds circuits, saved state, job records and every report. It survives restarts.
Circuit	A named gate program on 1 to 29 qubits. You create it (`q.circuit new`), add gates (`q.circuit add`), then run it (`q.circuit run`) to get measurement counts.
Selected circuit	Most `q.circuit` commands act on the circuit you created or selected last. Switch with `q.circuit select NAME`.
Sequence	A list of commands run one after another as a background job. Presets and folder files run as sequences. `q.sequence status` shows progress.
Tranche	A separate register for larger structured states (up to 29 qubits) with its own commands (`q.tranche ...`).
Procedural grid	A very large *described* model (for example 1000 × 1000 addresses). Each address's state comes from a formula. It is not stored as one giant vector. Commands: `q.prepare`, `q.migrate`, `q.gate`, `q.state`, `q.measure`, `q.verify`.

Concept	What it means
Capacity and auto-prune	The workspace keeps a limited number of circuits (64 by default). `q.circuit capacity` shows the limit. Circuits created by presets are pruned automatically when space is needed. Your own circuits are never deleted silently.
State and memory display	`q.circuit status`, `q.tranche status` and `q.status` show qubit counts, memory use and the current job. `q.resource headroom` shows how much memory is free for simulation.

---

3. Command reference (by family)

Arguments use `name=value`. Lists use `[a,b]`. Text with spaces goes in quotes. Angles are in radians, and expressions such as `pi/4` or `1/sqrt(2)` are allowed.

Circuits

Command	What it does	Example
q.circuit new	Create a circuit	`q.circuit new demo 2`
q.circuit add	Add a gate (angles first, then qubits)	`q.circuit add h 1`
q.circuit input	Set the starting basis state	`q.circuit input 0`
q.circuit show	Show the gate list	`q.circuit show`
q.circuit run	Simulate and sample measurement counts	`q.circuit run shots 500 basis z`
q.circuit result	Show the last run's results	`q.circuit result`
q.circuit state	Amplitude of one basis state	`q.circuit state 3`
q.circuit spin	Bloch vector of one qubit	`q.circuit spin 0`
q.circuit export	Export the circuit definition	`q.circuit export`
q.circuit list	List circuits in the workspace	`q.circuit list`
q.circuit select	Make a circuit current	`q.circuit select demo`
q.circuit preset	Build and run a ready-made circuit (bell, ghz, qft). Add `run=false` to only build it	`q.circuit preset bell 2 as demo_bell`
q.circuit status	Current circuit and job status	`q.circuit status`
q.circuit capacity	Show the circuit capacity	`q.circuit capacity`
q.circuit help	Circuit help	`q.circuit help`

Queries (on the selected circuit's last run)

Command	What it does	Example
q.query amplitudes	List amplitudes	`q.query amplitudes max=16`
q.query probabilities	List probabilities	`q.query probabilities max=16`
q.query marginal	Probabilities for some qubits	`q.query marginal qubits=[0] basis=z`
q.query reduced-density	Reduced density matrix	`q.query reduced-density qubits=[0]`
q.query purity	Purity of a subsystem	`q.query purity qubits=[0]`

Command	What it does	Example
q.query entropy	Entanglement entropy	`q.query entropy qubits=[0] base=2`
q.query expectation	Expectation of a Pauli string	`q.query expectation pauli="Z0 Z1"`
q.query concurrence	Two-qubit entanglement measure	`q.query concurrence qubits=[0,1]`
q.query fidelity	Overlap with a given state	`q.query fidelity state=[1/sqrt(2),0,0,1/sqrt(2)] convention=squared`

Tranche register

Command	What it does	Example
q.tranche prepare	Create a register	`q.tranche prepare 4 zero`
q.tranche gate	Apply a gate	`q.tranche gate h 0`
q.tranche state	Read one amplitude	`q.tranche state 0`
q.tranche spin	Bloch vector of a qubit	`q.tranche spin 0`
q.tranche measure	Sample measurements	`q.tranche measure 0 basis z shots 100`
q.tranche status	Register status	`q.tranche status`
q.tranche export	Export the register	`q.tranche export`

Procedural grid

Command	What it does	Example
q.prepare grid	Describe a large grid model	`q.prepare grid 1000 1000`
q.migrate	Move the model in time	`q.migrate t 1`
q.gate	Apply a gate to every address	`q.gate h all`
q.state	State at one address	`q.state 12345`
q.measure	Sample one address	`q.measure 12345 basis z shots 100`
q.verify	Rule check / CPU-GPU sample check	`q.verify rule`
q.export	Export the model descriptor	`q.export descriptor`
q.status	Full session status	`q.status`

Resource planning

Command	What it does	Example
q.resource memory	Memory needed for n qubits	`q.resource memory n=29 representation=statevector dtype=complex128 units=binary`
q.resource max-n	Largest n for a memory budget	`q.resource max-n memory-mib=512 representation=statevector dtype=complex128 overhead-mib=0`
q.resource shots	Shots needed for a target accuracy	`q.resource shots epsilon=0.01 delta=0.05 bound=hoeffding-two-sided`
q.resource sampling	Error bars of a probability estimate	`q.resource sampling probability=0.5 shots=1024 metric=standard-error`
q.resource headroom	Free memory for simulation	`q.resource headroom`

Math calculators

Command	What it does	Example
q.math normalize	Normalize a state	`q.math normalize amplitudes=[1,i]`
q.math probabilities	Born probabilities	`q.math probabilities amplitudes=[sqrt(3)/2,i/2]`
q.math bloch	Bloch vector of a qubit state	`q.math bloch amplitudes=[1/sqrt(2),i/sqrt(2)]`
q.math density	Density matrix	`q.math density amplitudes=[1/sqrt(2),i/sqrt(2)]`
q.math fidelity	Fidelity of two states	`q.math fidelity a=[1/sqrt(2),1/sqrt(2)] b=[1/sqrt(2),-1/sqrt(2)] convention=squared`
q.math gate-matrix	Matrix of a gate	`q.math gate-matrix gate=h`
q.math tensor	Tensor product	`q.math tensor left=H right=I2`
q.math eigen	Eigenvalues and eigenvectors	`q.math eigen operator=X`
q.math expectation	Expectation value	`q.math expectation state=[sqrt(3)/2,1/2] operator=Z`

Physics models, lessons, noise, error correction, compilation, algorithms

Command	What it does	Example
q.model spectrum	Energy levels of a small Hamiltonian	`q.model spectrum hamiltonian="-1*Z0" n=1`
q.model evolve	Time evolution	`q.model evolve n=1 initial=[1/sqrt(2),1/sqrt(2)] hamiltonian="0.5*Z0" time=pi/2 hbar=1`
q.lesson	Guided explanation	`q.lesson amplitude-vs-probability example=[1/sqrt(2),i/sqrt(2)]`
q.noise apply	Apply a noise channel	`q.noise apply state=[1,0] channel=bit-flip p=0.1`
q.noise zne	Zero-noise extrapolation	`q.noise zne scales=[1,2,3] expectations=[0.8,0.7,0.6] model=linear`
q.qec repetition3	3-qubit repetition code	`q.qec repetition3 encode state=[1/sqrt(2),1/sqrt(2)]`
q.qec parameters	Code parameters	`q.qec parameters n=7 independent-stabilizers=6`
q.compile identity	Check two gate lists are equal	`q.compile identity left="h 0;x 0;h 0" right="z 0" n=1 equivalence=exact`
q.compile count	Count gates	`q.compile count n=2 gates="h 0;t 0;cx 0 1;tdg 1" counting=logical`
q.algorithm phase-estimation	Exact phase estimation demo	`q.algorithm phase-estimation counting-qubits=3 unitary="p 2.356194490192345 3" eigenstate=[0,1] power-map="qj-controls-U^(2^j)" inverse-qft=include-final-swaps`

Sequences, presets, history, help

Command	What it does	Example
q.preset list	List the 241 presets	`q.preset list`

Command	What it does	Example
q.preset show	Show one preset	`q.preset show qc-097`
q.preset run	Run one preset	`q.preset run qc-097`
q.sequence status	Progress of the current job	`q.sequence status`
q.folder help	Folder-run help	`q.folder help`
q.history list	Recent reports	`q.history list`
q.help	List all commands	`q.help`

---

4. Presets

SpinDoctor includes **241 presets** (`qc-001` … `qc-241`): ready-made, validated command lists.

- `q.preset list` lists them all with their categories. `q.preset show qc-097` shows what one does.
- `q.preset run qc-097` runs a preset as a sequence. Shot-based presets default to **1,024 shots**. Many accept overrides; see `q.preset show`.
- **Categories (number of presets):** Procedural grid (9), Tranche controls (12), Single-qubit gates (40), Two-qubit gates and entanglement (35), Multi-qubit circuits (20), Algorithm demonstrations (15), State and matrix mathematics (22), Joint-state analysis (14), Noise and mitigation (12), Error-correction demonstrations (10), Compilation and gate synthesis (10), Memory and resource estimates (12), Alternative computation models (8), Guided lessons and article corrections (8), Circuit presets and controls (10), Large structured-register demonstrations (3), Algorithm extensions (1).
- **qc-198 is unavailable on purpose.** It needs a Clifford+T synthesizer (such as gridsynth) that SpinDoctor does not include, so it stops with "Preflight: Preset is unavailable" rather than inventing a result.

---

5. Reports

- **Every command produces a report automatically**, including errors and status checks.
- **TXT** is the exact, complete record. **PDF** is a readable version with graphs, circuit diagrams, histograms and Bloch spheres. **CSV** is a typed table of the same data for spreadsheets.
- **Where they go:** the native archives stay inside `SpinDoctor\workspace` (folders `quantum-command-reports`, `sequence-reports`, `sequence-jobs`). Downloads made from the interface go to your browser's download folder. Folder runs with `downloads=true` (the default) download their reports automatically.
- Exporting a report never recalculates anything. It reads the saved result.
- `q.folder status` and `q.sequence status` polls now save a small report (3.1.2). Use `q.status` when you want a full snapshot saved.

---

6. Folder ingestion (`q.folder`)

Run every input file in a folder, one file after another:

```
q.folder run path="C:/Users/me/Desktop/inputs" downloads=false verify=false
q.folder status
q.folder cancel
```

- **Supported files:** `.txt`, `.csv`, `.json`, `.qasm`, `.qasm2`, `.circuit`, `.pdf`. Other files (for example `.md`, `.docx`, `.png`) are **skipped** and listed in the job's `skippedFiles` (new in 3.1.2).
- **Top level only.** Subfolders are not read. Files run in strict character order (digits, then capitals, then `_`, then lowercase).
- **Limits:** 1–128 supported files per run; 4 MiB per file; 1–128 commands per file; 4,096 characters per command; text must be valid UTF-8.
- **The first failing file stops the job.** Files before it keep their results; files after it never run.
- **Verification:** by default (`verify=true`) each file is followed by an automatic CPU/OpenCL-GPU sample check of the procedural model before its success receipt is released. This needs a working OpenCL GPU. `verify=false` gives clearly labelled unverified receipts.
- While a folder job runs, only read-only commands (status, help, history) are accepted.
- Folders and files must be on a local drive (no network paths), and links (junctions or symlinks) are refused. OneDrive "online-only" placeholder files may also be refused as links. If you get a reparse-point error in a OneDrive folder, right-click the folder, choose **Always keep on this device**, and try again.

---

7. Input file formats (tiny examples)

.txt command file. Lines starting with `#` or `//` and blank lines are ignored. Every other line is one command. Prose and ``` fences are not allowed.

```
# Bell pair
q.circuit new filebell 2
q.circuit add h 0
q.circuit add cx 0 1
q.circuit run shots 256 basis z
```

.txt / .circuit text circuit. A file with no `q.*` lines is read as a circuit:

```
qubits 2
shots 512
h 0
cx 0 1
```

.csv command file. One column named `command`:

```
command
"q.math normalize amplitudes=[1,i]"
"q.math bloch amplitudes=[1/sqrt(2),i/sqrt(2)]"
```

.json. Either `{"commands": ["q.math eigen operator=Z"]}` or a circuit:

```
{"name": "jsonghz", "qubits": 3, "gates": [{"name": "h", "qubits": [0]}, {"name": "cx", "qubits": [0, 1]}], "shots": 256, "basis": "z"}
```

.qasm / .qasm2. OpenQASM 2.0:

```
OPENQASM 2.0;
include "qelib1.inc";
qreg q[1];
creg c[1];
h q[0];
measure q -> c;
```

.pdf. The PDF must either embed one `.txt/.csv/.json/.qasm` file, or contain the program text between `SPINDOCTOR-TXT-BEGIN` and `SPINDOCTOR-TXT-END` (or `-JSON-`, `-QASM-`, `-CSV-`). Scanned or prose PDFs are not guessed at.

Measurement CSVs (weather, gate, or circuit data) can also be uploaded in the interface and fitted with `q.input fit`. See `SpinDoctor-3.1.1-CSV-Input-Guide.txt`.

---

8. Limits

Item	Limit
Qubits per circuit / tranche	1–29
Shots per run	1–100,000 (presets default to 1,024)
Instructions per circuit	8,192
Gate angle	±1e12 radians
Folder files per run	1–128 supported files
File size	4 MiB
Commands per file / sequence	1–128
Command length	4,096 characters
Circuits in a workspace	64 by default, adjustable up to 1,024 (`q.circuit capacity`)
Folder verification samples	1–16,384,000,000 (default 10,240)
PDF export	512 pages, 32 MiB

---

9. Common errors and what they mean

Message (start)	Meaning / what to do
`Input sequences accept documented quantum commands only…`	A line in a command file is not a `q.*` command. It might be prose, a `` ``` `` fence, or `q.folder` / `q.input`. Remove it or make it a `#` comment.
`Text input is empty…`	The `.txt` file has only blank or comment lines.
`Text input line N is neither a q.* command nor a known gate…`	The file has no `q.*` commands, and line N isn't a gate either.
`Text circuit requires qubits N…`	A text circuit has gate lines but no `qubits N` line.

Message (start)	Meaning / what to do
`Input sequences contain 1..128 commands.`	Split the file into smaller files.
`Each sequence entry is one bounded terminal command.`	A command is longer than 4,096 characters.
`Submit ordinary commands as an array; unquoted macro separators…`	Put each command on its own line. `;` is only allowed inside quotes.
`Input sequences do not delete/reset existing user state.`	`delete` and `reset` are not allowed in files. Type them yourself.
`Folder files are limited to 4 MiB each…`	The file is too big.
`Folder mode supports at most 128 input files per job.`	Split the folder.
`No supported JSON/QASM/QASM2/TXT/CIRCUIT/CSV/PDF input files were found…`	The folder has no readable files. The message names a skipped file.
`Preflight: Preset is unavailable: …`	The preset needs a tool SpinDoctor doesn't have (qc-198).
`Unknown circuit action.`	Misspelled `q.circuit` sub-command. See `q.circuit help`.
`Circuit name already exists. Select it or delete it first.`	Use a new name or `q.circuit delete NAME`.
`Gate angle exceeds +/-1e12 radians.`	Use a smaller angle.
`Unable to translate bytes […]`	The file isn't valid UTF-8. Re-save it as UTF-8.
`Reparse-point … not accepted.`	The path is a link or junction, or an online-only OneDrive file.
`A sequence owns the calculation context…`	A job is running. Wait, or use `q.sequence cancel` / `q.folder cancel`.

10. Troubleshooting

- **BOOT.cmd closes immediately or says the carrier is missing.** Keep the `SpinDoctor` folder intact. `cartridge\SpinDoctor-3.1.2.tiff` and `harness\` must be next to `BOOT.cmd`. Run `VERIFY.cmd` to check.
- **Browser page doesn't open.** Copy the address shown in the console window into your browser. Don't close the console.
- **"Circuit name already exists"** when you run a file again: use a new circuit name, delete the old one with `q.circuit delete NAME`, or clear preset-made circuits with `q.circuit prune presets`. (`RESET_CURRENT.cmd` does not erase anything. Use `q.reset` to reset the procedural model.)
- **Folder run waits at verification, or fails it.** You need a working OpenCL GPU driver. Use `verify=false` if you don't need the GPU sample check.
- **Out of memory / budget errors.** Large dense circuits need gigabytes from about 26 qubits up (each extra qubit doubles the memory). Check `q.resource memory n=…` and `q.resource headroom` first.
- **Many large reports.** Every command keeps a full report, and long folder runs can use hundreds of MB to a few GB in `SpinDoctor\workspace`. Move or delete old workspaces you no longer need.

11. What changed in 3.1.2

- **Folder runs report skipped files.** Unsupported files are listed in `skippedFiles` / `skippedFileCount` instead of being ignored silently. A folder with no readable files names one of the skipped files in its error.
- **`.qasm2` and `.circuit` files** are now accepted by `q.folder`. The parser already supported them.
- **Clearer text errors.** Empty `.txt` files and prose-only `.txt` files now say what is wrong instead of reporting a missing `qubits N` line.
- **Compact polling reports.** Each `q.folder status` / `q.sequence status` used to archive a full session snapshot (up to about 1 MB each). It now archives a few KB. The live response is unchanged.
- Version bumped to 3.1.2 everywhere, new tests added, and this hand guide added.

---

12. Honest limits

- **SpinDoctor is a classical simulator.** It runs on your CPU (and GPU for verification). It is not a quantum computer and gives no quantum speed-up.
- **Qubit scaling is exponential.** A general (dense) state of n qubits needs 2^n complex numbers, so 29 qubits needs 8 GiB. Each extra qubit doubles that. Structured states (product states, GHZ, the procedural grid) can be much larger because they are stored compactly. That does not make arbitrary circuits cheap.
- **The procedural grid is a described model.** Its "trillions of addresses" are computed on demand from a formula, not stored or measured physically.
- **Sampling has noise.** Histograms from shots are random samples. Expect normal statistical variation. For example, a 50/50 outcome measured with 1,000 shots typically varies by about ±1.6 percentage points (one standard deviation).
- **Fits are not laws.** CSV equation fitting is classical least squares on your data. It does not prove a physical law or predict unseen data.
- **Verification has a scope.** `q.verify` checks sampled points of the procedural model on CPU and GPU. It does not certify every calculator answer or unsampled state.